502094 - Software Deployment, Operations And Maintenance

Module 3: Server Access, Tunneling & Remote DevOps Tools

Lecturer: MSc. Mai Van Manh

Email: maivanmanh@tdtu.edu.vn

Module Overview

- **Goal:** Access, manage, and expose servers and dev environments safely.
- Focus on **Ubuntu 22.04** in a backend/DevOps context.
- Three major themes:
  - Exposing local development services to the internet.
  - Working effectively and securely with **SSH**.
  - Using **CLI tools** (tmux, ufw) for robust remote work.

Why Server Access & Tunneling Matter

- Modern backend/DevOps work is **remote-first**:
  - Cloud VMs, containers, managed services.
- You must be able to:
  - Reach a server securely.
  - Expose internal services safely (for demos, webhooks, testing).
  - Maintain long-lived sessions and protect the server.
- Poor practices (e.g. exposing everything on the internet) lead to:
  - **Security incidents**, **downtime**, and **data loss**.

Exposing Local Websites to the Internet

- Often we run:
  - **Local dev servers** (e.g. `npm run dev`, `flask run`, `php -S`).
- We want to share them with:
  - Teammates, testers, or external services (e.g. **webhooks**, payment gateways).
- Techniques we will cover:
  - VS Code port forwarding.
  - **localtunnel**.
  - **ngrok**.
  - Home router port forwarding.
  - **Cloudflared** tunnel.

VS Code Port Forwarding (Concept)

- VS Code can forward ports when using:
  - **Remote - SSH** or **Dev Containers**.
- Idea:
  - Your app listens on a port on the **remote machine** (e.g. `localhost:3000`).
  - VS Code tunnels that port back to your local machine.
- Benefits:
  - Seamless debugging and editing.
  - No need to expose ports on the public internet.

VS Code Port Forwarding (Example)

- Scenario:
  - Your Node.js app is running on a remote Ubuntu VM:

```
node server.js  # listening on http://localhost:3000
```

- In VS Code (Remote-SSH):
  - Open the **Ports** panel.
  - Add a forwarded port:
    - **Remote port:** `3000`
    - **Local port:** `3000` (or any free port)
- You can now open:
  - `http://localhost:3000` in your **local browser** to reach the remote app.

localtunnel (Concept)

- **localtunnel** creates a **public URL** that forwards traffic to a local port.
- Useful when:
  - You are behind NAT / firewall.
  - You cannot configure router port forwarding.
- Typical use cases:
  - Testing **webhooks** from Stripe, GitHub, etc.
  - Letting teammates open your dev UI temporarily.

localtunnel (Installation & Usage)

- Install globally with **npm**:

```
npm install -g localtunnel
```

- Run your local dev server:

```
npm run dev   # e.g. Next.js on http://localhost:3000
```

- Start localtunnel:

```
lt --port 3000 --subdomain tdtu-dev-demo
```

- Output will include a URL like:

```
your url is: https://tdtu-dev-demo.loca.lt
```

localtunnel: Considerations

- Pros:
  - Very easy to use.
  - No router configuration required.
- Cons / Risks:
  - URL is public: **anyone** who knows it can access your service.
  - Performance depends on the tunnel provider.
  - Do **not** expose production databases or admin panels this way.
- Best practice:
  - Use **demo data**, limit permissions, and stop the tunnel when finished.

ngrok (Concept)

- **ngrok** is a popular tunneling service for:
  - Exposing local HTTP / HTTPS / TCP services.
- Features:
  - Public URLs (temporary or reserved).
  - Request inspection UI.
  - Authentication and access control features.
- Common for:
  - Webhook development.
  - Quick demos with nicer URLs.

ngrok (Basic Usage)

- Install ngrok by following instructions at https://ngrok.com/download.
- Start your local app:

```
npm run dev  # listening on http://localhost:3000
```

- Start ngrok:

```
ngrok http 3000
```

- You will see output like:

```
Forwarding  https://d1b2c3d4e5.ngrok.io -> http://localhost:3000
```

ngrok: Security & Configuration

- Create an account and add your **authtoken**:

```
ngrok config add-authtoken YOUR_TOKEN_HERE
```

- Can configure:
  - Reserved subdomains.
  - Basic auth:

```
ngrok http 3000 --basic-auth="user:strongpassword"
```

- Still **not for production**:
  - Use only for temporary testing.
  - Treat it as exposing your local host to the public internet.

Home WiFi Router Port Forwarding (Concept)

- Traditional way to expose a local server:
  - Configure **port forwarding** on your router.
- Example:
  - Public IP: `203.0.113.20`
  - Internal server: `192.168.1.50`, port `3000`.
  - Router rule: forward **TCP 8080** → `192.168.1.50:3000`.
- Then others can access:
  - `http://203.0.113.20:8080`.

Home Port Forwarding: Risks & Use Cases

- Useful for:
  - Personal hobby projects.
  - Hosting services for yourself or a small group.
- Major risks:
  - Exposes your home network to the internet.
  - Attackers constantly scan common ports.
- Best practices:
  - Use **strong passwords**.
  - Restrict by IP if possible.
  - Prefer **VPN** or tunnels (ngrok, Cloudflared) for temporary access.

Cloudflared Tunnel (Concept)

- **Cloudflared** is Cloudflare's tunneling agent.
- Connects your local service to Cloudflare's network:
  - Without opening inbound ports on your router or VM.
- Use cases:
  - Serve local apps via `https://yourname.example.com`.
  - Hide your origin IP and rely on Cloudflare's edge.

Cloudflared Tunnel (Quick Demo)

- Install on Ubuntu:

```
sudo apt update
sudo apt install cloudflared
```

- Run a quick tunnel to a local app on port 3000:

```
cloudflared tunnel --url http://localhost:3000
```

- Output will show a **public URL**, for example:

```
https://dev-app-tdtu.trycloudflare.com
```

- For production, you would create a named tunnel, connect it to a domain on Cloudflare, and restrict access (e.g. via Cloudflare Access).

Exposing Local Services: Trade-offs

- **Localtunnel / ngrok / Cloudflared**:
  - Easy setup, temporary, ideal for dev and demos.
- **Router port forwarding**:
  - More permanent, but higher security risk.
- **VS Code port forwarding**:
  - Local-only, ideal for **developer** access, not external users.
- Always ask:
  - Do I really need this to be reachable from the public internet?
  - Can I authenticate or restrict who can access it?

Working with SSH

- **Secure Shell (SSH)** is the standard protocol for:
  - Secure remote login.
  - Command execution.
  - File transfer and tunneling.
- In DevOps, SSH is used to:
  - Manage Linux servers.
  - Deploy and debug applications.
  - Automate tasks.
- We will cover:
  - `Server-side`: enabling SSH, `Client-side`: keys, config, tunnels, file transfer.

Enabling SSH Server on Ubuntu 22.04

- Install the **OpenSSH server**:

```
sudo apt update
sudo apt install openssh-server
```

- Check and enable the service:

```
sudo systemctl status ssh
sudo systemctl enable --now ssh
```

- Verify that the server is listening:

```
sudo ss -tlnp | grep sshd
```

- Now the server accepts SSH connections on port **22** (by default).

SSH Server: Basic Hardening

- Principles:
  - **Least privilege**: avoid logging in directly as `root`.
  - Prefer **key-based authentication** instead of passwords.
- Quick checks:

```
sudo nano /etc/ssh/sshd_config
```

  - Set:
    - `PermitRootLogin no`
    - `PasswordAuthentication no` (after keys are working)
- Reload SSH:

```
sudo systemctl reload ssh
```

Connecting with SSH (Password Auth)

- Basic syntax:

```
ssh USERNAME@SERVER_IP
```

- Example:

```
ssh ubuntu@203.0.113.50
```

- First connection:
  - You will see a message about the **host key fingerprint**.
  - Type `yes` to trust and store it in `~/.ssh/known_hosts`.
- Then enter the **password** for the user.
- This is convenient but not ideal for automation or security.

SSH Key-Based Authentication (Concept)

- Uses **public-key cryptography**:
  - **Private key** stays on your local machine.
  - **Public key** is copied to the server.
- Benefits:
  - Stronger security than passwords.
  - Enables **passwordless login**.
  - Works well with automation and tools (Ansible, Git, CI/CD).

Generating SSH Keys on Linux/macOS

- Use the `ssh-keygen` command:

```
ssh-keygen -t ed25519 -C "student@tdtu.edu.vn"
```

- You will be asked:
  - File to save the key (default: `~/.ssh/id_ed25519`).
  - A **passphrase** (recommended for security).
- This creates:
  - Private key: `~/.ssh/id_ed25519`
  - Public key: `~/.ssh/id_ed25519.pub`
- Never share the **private key** with anyone.

Generating SSH Keys on Windows

- Options:
  - **PowerShell** or **Git Bash**.
- Example using PowerShell:

```
ssh-keygen -t ed25519 -C "student@tdtu.edu.vn"
```

- Keys are usually stored in:
  - `C:\Users\YourName\.ssh\id_ed25519`
- If using **WSL**, keys are stored in the Linux home directory, just like on Ubuntu.

Setting Up Passwordless SSH Login

- Suppose your public key is in `~/.ssh/id_ed25519.pub`.
- Use `ssh-copy-id` (Linux/macOS):

```
ssh-copy-id ubuntu@203.0.113.50
```

- This appends your public key to:
  - `/home/ubuntu/.ssh/authorized_keys` on the server.
- Now you can log in without typing the password:

```
ssh ubuntu@203.0.113.50
```

Passwordless SSH: Manual Setup

- If `ssh-copy-id` is not available:
  - Show the public key:

    ```
    cat ~/.ssh/id_ed25519.pub
    ```

  - Copy the whole line.
- On the server:

  ```
  mkdir -p ~/.ssh
  chmod 700 ~/.ssh
  echo "ssh-ed25519 AAAA... student@tdtu.edu.vn" >> ~/.ssh/authorized_keys
  chmod 600 ~/.ssh/authorized_keys
  ```

- Test that you can log in without a password.

Managing the SSH Config File

- File: `~/.ssh/config`
- Lets you define **friendly names** for servers.
- Example:

```
Host prod-server
  HostName 203.0.113.50
  User ubuntu
  IdentityFile ~/.ssh/id_ed25519
  Port 22
```

- Now you can simply run:

```
ssh prod-server
```

- Helps avoid typos and keeps connection options consistent.

Remote Development with VS Code & SSH

- Tools: VS Code + **Remote - SSH** extension.
- Workflow:
  - Configure SSH access (keys, config file).
  - In VS Code, choose:
    - **Remote Explorer → SSH Targets → Connect to Host**.
  - VS Code opens a remote window:
    - File explorer shows the remote filesystem.
    - Integrated terminal runs on the server.
- Benefits:
  - Develop “locally” while code runs **on the server**.
  - Great for debugging issues that only appear in the server environment.

Remote Execution & Automation with SSH

- You can run a single command remotely:

```
ssh prod-server "uname -a"
ssh prod-server "sudo systemctl restart nginx"
```

- This is useful for:
  - Quick checks (uptime, disk space).
  - Simple one-off deployments or restarts.
- In automation (e.g. shell scripts, CI jobs):
  - Combine with SSH keys for passwordless, scripted tasks.

File Transfer with scp

- **scp** (secure copy) uses SSH to copy files.
- Upload a file:

```
scp ./main.go ubuntu@203.0.113.50:/var/www/app/
```

- Download a file:

```
scp ubuntu@203.0.113.50:/var/log/nginx/error.log ./logs/
```

- Useful for:
  - Uploading build artifacts.
  - Downloading logs for analysis.
- For large or frequent transfers, **rsync** is more efficient.

Syncing Directories with rsync

- **rsync** transfers only changed files.
- Basic deploy-like example:

```
rsync -avz ./dist/ ubuntu@203.0.113.50:/var/www/app/
```

  - `-a`: archive mode (preserve permissions, etc.).
  - `-v`: verbose.
  - `-z`: compress during transfer.
- Mirror local to remote (delete files removed locally):

```
rsync -avz --delete ./dist/ ubuntu@203.0.113.50:/var/www/app/
```

- Great for repeatable, efficient deployments.

SSH Tunneling (Concept)

- SSH can **forward ports** securely.
- Two main types:
  - **Local forwarding (`-L`)**:
    - Connect local port → remote host:port.
  - **Remote forwarding (`-R`)**:
    - Expose local host:port → port on the remote server.
- Use cases:
  - Securely access internal services.
  - Expose your local dev server via a remote public VM.

Local Port Forwarding: Example

- Remote server runs a web UI on `localhost:9000` (only accessible inside).
- You want to open it from your laptop on port **9000**.
- Use:

```
ssh -L 9000:localhost:9000 ubuntu@203.0.113.50
```

- While the SSH session is open:
  - Open `http://localhost:9000` on your laptop.
  - Traffic goes through the SSH tunnel to the remote server.

Remote Port Forwarding: Example

- You run a dev app locally on `localhost:3000`.
- Remote VM (with public IP) should expose it on port **8080**.
- Use:

```
ssh -R 0.0.0.0:8080:localhost:3000 ubuntu@203.0.113.50
```

- While the SSH session is open:
  - Others can open `http://203.0.113.50:8080`.
  - Requests are forwarded to your local dev app.
- Combine with **ufw** to restrict who can access port 8080.

Choosing a Tunneling Technique

- **Local (`-L`) forwarding**:
  - Access a remote internal service from your local machine.
- **Remote (`-R`) forwarding**:
  - Make your local service reachable via a remote server.
- Higher-level tools built on tunnels:
  - VS Code port forwarding.
  - localtunnel, ngrok, Cloudflared.
- Always limit exposure and close tunnels when done.

CLI Tools When Working Remotely

- Remote work challenges:
  - Unstable network connections.
  - Need for **persistent sessions**.
  - Need to control network access to services.
- Two important tools:
  - **tmux**: terminal multiplexer.
  - **ufw**: uncomplicated firewall.

The Problem Without tmux

- Scenario:
  - You SSH into a server and run a long task:

    ```
    npm run build
    ```

  - Your laptop sleeps or your Wi-Fi disconnects.
  - SSH session closes → process stops.
- This is common during:
  - Database migrations.
  - Long builds or backups.
- We need a way to keep processes running **independently** of the SSH session.

tmux (Concept)

- **tmux** = terminal multiplexer.
- It allows you to:
  - Create **persistent sessions**.
  - Split terminal into **multiple panes**.
  - Detach from a session and reattach later.
- Great for:
  - Long-running tasks.
  - Monitoring logs and services in parallel.

tmux: Basic Workflow

- Install tmux:

```
sudo apt update
sudo apt install tmux
```

- Start a new session:

```
tmux new -s dev-session
```

- Inside tmux:
  - Run long tasks, e.g.:

```
npm run dev
```

- Detach from the session:
  - Press `Ctrl + b`, then `d`.
- Later, reattach:

tmux: Practical Example

- A typical DevOps workflow:
  - Window 1: run the application:

    ```
    npm run dev
    ```

  - Split another pane (Ctrl + b, then `%` or `"`).
  - Pane 2: tail logs:

    ```
    tail -f /var/log/nginx/access.log
    ```

- Even if SSH disconnects:
  - The tmux session (and your processes) keep running.
  - You reconnect and reattach to continue.

Firewall Basics with ufw

- **ufw** = Uncomplicated Firewall (front-end for `iptables`/`nftables`).
- Purpose:
  - Control which **incoming** connections are allowed.
  - Reduce attack surface of your server.
- Principle:
  - **Default deny**: block everything, then open only required ports.

Configuring ufw on Ubuntu

- Check status:

```
sudo ufw status verbose
```

- Before enabling, allow SSH:

```
sudo ufw allow OpenSSH
# or explicitly
sudo ufw allow 22/tcp
```

- Enable ufw:

```
sudo ufw enable
```

- Now only allowed ports can be reached from the network.

Allowing Web Traffic with ufw

- For a typical web server:
  - Allow HTTP (80) and HTTPS (443):

```
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
```

- Review rules: `sudo ufw status numbered`
- Remove a rule (example: rule 3): `sudo ufw delete 3`
- Combine ufw with:
  - SSH key-based auth.
  - Limited tunnels and temporary ports.

Putting It All Together

- For a small backend project in the cloud:
  - Use **SSH keys** and `~/.ssh/config` to manage access.
  - Use **tmux** for long-running processes and monitoring.
  - Use **ufw** to allow only SSH, HTTP, and HTTPS.
  - For external demos:
    - Prefer **ngrok**, **localtunnel**, or **Cloudflared** over permanent port forwarding.
  - Use **rsync/scp** for deployments and log collection.

Checklist for Your Next Server

- Installed and enabled **openssh-server**.
- SSH key-based authentication configured; root login disabled.
- `~/.ssh/config` entries for quick access.
- **tmux** installed and used for long tasks.
- **ufw** enabled with only required ports open.
- Tunneling tools (ngrok/localtunnel/Cloudflared) understood and used carefully.
- Never expose sensitive services directly to the public internet.